

技術簡報 Technical Overview

xConsole
模型主控台

統一記憶體 AI 機的模型主控台：准入演算法、雙引擎換模型、安全架構、維運與已知限制

云碩科技股份有限公司　2026 年 10 月　main 分支（ 2026 年 10 月 22 日 ）

這份簡報的讀法：每個模組都標出做到哪裡

狀態標記全本一致；數字一律對得到測試輸出、程式常數與威脅模型

已實作且有測試

程式完成並有自動測試，且已在云碩自有機群（兩台 AMD AI 395 、一台 Jetson AGX Thor ）的正式環境運作。

已實作待實機驗證

程式與自動測試完成，還沒在真實硬體上跑過。目前屬於這一類的是 DGX Spark 的實機納管。

規劃中

列在威脅模型缺口清單，尚未實作，集中放在最後的已知限制與路線圖頁，不寫成功能。

數字口徑

測試數字是 2026 年 10 月 22 日在 .27 實際執行的通過項計數，共 214 項，另有 12 項真機冒煙測試。記憶體與速度數字取自自有機群的量測紀錄，不是規格值；每個數字的出處寫在備忘稿。

內容順序

1 風險與架構

2 記憶體准入

3 換模型與雙引擎

4 模型庫與 API 串接

5 安全架構

6 憑證、稽核、報告

7 證據與規格

8 限制、路線圖、導入

做到的寫做到，沒做到的寫出缺口與期限。

為什麼需要：統一記憶體機的失效方式跟一般伺服器不同

云碩自有機群的實際紀錄

記憶體是共用的　AMD AI 395 把 96 GiB 切給 GPU ，作業系統只剩 31 GiB ； Jetson 與 DGX Spark 則是 GPU 和系統搶同一池。傳統監控只看顯存，在這些機器上會盯錯數字。

失效不會報錯　記憶體耗盡時引擎多半不會回錯誤，而是系統開始大量使用置換空間，整台越來越慢，最後連 SSH 都進不去，只能到現場重開。

用量會在載入後才長大　模型剛載入時用量不高，處理長提示時上下文快取才撐開；只看當下用量放行，稍後就會出事。

自有機群紀錄

2026 年 10 月 2 日： AI 395 系統記憶體可用 1 GiB / 31 GiB ，置換空間用掉 25 GiB ，負載 7.7 ，每段處理 67 秒。

更早以前：記憶體耗盡導致整台卡死，不只一次需到機房重開。

2026 年 10 月 20 日：裝上 xConsole 後再載一顆 64 GiB 模型，試算剩 3.2 GiB ，低於 15 GiB 底線而擋下。

要防的是「載下去之後最壞剩多少」，不是「現在用了多少」。

系統全景：誰在用、能做什麼、跑在哪裡

已實作且有測試 AI 395 與 Jetson 已在正式環境　已實作待實機驗證 DGX Spark 待實機

使用

誰在用
單位內部

寫程式的同仁 | 研究團隊 | 資訊室 | 主管 | 稽核

主控

主控台
一個網頁管全部

xConsole　一台也管，十台也管

載入前先試算 | 網頁上換模型 | IDE一頁串好 | 每月使用報告

能力

能做什麼
從載入到報告

- 記憶體准入 — 放不下就擋
- 角色欄位 — 一眼看缺什麼
- 換模型 — 兩種引擎
- 模型庫 — 下載與刪除
- API串接 — 8種範例
- HTTPS憑證 — 網頁上換
- 使用報告 — 給主管月報
- 操作紀錄 — 保存400天

機器

跑在哪裡
單位自己的AI機

AMD AI 395 | Jetson / MIC-743 | DGX Spark | 多卡伺服器 | 資料不出內網

元件與執行方式：三個程序，全部以一般使用者執行

不需要 root 、不需要容器；依賴只有 cryptography 一套

元件	做什麼	執行方式	資源上限
主控台	網頁介面與 API 、准入彙整、憑證管理、稽核與取樣、使用報告	systemd 使用者服務，埠 9400	記憶體 512 MiB 、CPU 40%
節點代理	每台一個：讀硬體、准入試算、啟停服務、換模型、模型庫、下載	systemd 使用者服務，埠 9401	記憶體 256 MiB 、CPU 25%
推論引擎	Strata （ Qwen3.8 家族）或 llama.cpp （其他 GGUF ），由代理產生設定與服務	systemd 使用者服務，埠依欄位	由准入試算把關
資料	SQLite （稽核、取樣、登入票證）、設定檔、憑證與主金鑰	主控台的 ~/.xconsole ，目錄 0700 、檔案 0600	取樣與稽核保存 400 天

技術選擇　主控台與代理只用 Python 標準函式庫的 HTTP 伺服器，加上 cryptography 處理憑證與加密；前端是無框架的原生 JavaScript ，沒有建置步驟。依賴少， SBOM 只有四個套件，弱點面小。

依賴越少，要追的弱點越少。

部署型態：單機附在 AI 機上，多機選一台當主控台

已實作且有測試 單機與多機都在用　已實作且有測試 三台機器正式納管

單機模式
賣一台時附上

- **同一台AI機**：主控台與節點代理
- **第一次開機**：就是HTTPS
- **開瀏覽器就用**：不必另備管理機

多機模式
機器變多時

- **主控台**：選一台裝
- **節點代理**：每台各裝一個
- **互相驗證**：各機憑證經SSH取回
- **每5分鐘**：取樣各機狀態

擴充
不必重做

各機設定不用重做　加一台 = 裝代理再納管　總覽一頁看完

資源上限
不壓垮機器

主控台記憶體512 MiB　節點代理256 MiB　CPU使用量設上限

單機　多機　擴充　上限

三種記憶體架構分開計算

已實作且有測試 unified_split 、 unified_single 、 discrete

已實作待實機驗證 DGX Spark 實機

AMD AI 395 切兩塊	96 GiB給GPU	系統只剩31 GiB	系統底線15 GiB	切兩塊
Jetson / DGX 單一池	GPU與系統搶同一塊	合併計算	底線12 GiB	單一池
多卡伺服器 每卡各自算	每張卡的顯存	系統記憶體另算	顯存1 / 系統8 GiB	多卡
每次都算 啟動與換模型前	載下去之後最壞剩多少，低於底線就擋			試算

准入演算法：每次啟動、換模型、調大上下文前都會跑

已實作且有測試 agent/test_admit.py 15 項：三種架構、 KV 兩種量級、底線邊界、滑動視窗乘並行

輸入 各池可用量（由硬體量測）、欄位的預估需求（權重 + 常駐量）、最壞情況的上下文快取 = 每 token 位元組 × 上下文 × 並行數。

算回自己 重啟正在跑的欄位時，先把它目前占的量加回可用量；跨引擎切換時，舊引擎是另一個服務，另以 extra_free 算回。

判斷 任何一池在載入後低於底線就拒絕；距底線 4 GiB 內放行但提出警告。

輸出 allow 、 reasons （白話說明哪一池差多少）、 warnings 、 kv_note ；管理者可選擇強制執行，系統留下稽核紀錄。

架構	預設底線
unified_split	系統 15 GiB 、顯存 2 GiB
unified_single	單一池 12 GiB
discrete	系統 8 GiB 、顯存 1 GiB

擋下來時說得出差多少，管理者才知道要停哪個、調哪裡。

上下文快取怎麼估：兩種引擎、差好幾個數量級

已實作且有測試 Strata 以實測校正； llama.cpp 由 GGUF 檔頭計算，真機冒煙測試驗證過讀取

Strata（ Qwen3.8-Flash-Next ） GDN 線性混合注意力， int8 下每 token 只要 1,056 位元組。實測把上下文由 32,768 調到 131,072 ，載入後剩餘顯存由 33,026 ~ 33,416 MiB 變為 33,094 MiB ，幾乎不變。

llama.cpp（ GGUF ） 直接讀檔頭的層數、每層 KV 頭數與頭維度：每 token 位元組 = Σ 每層 KV 頭數 ×(K + V)×2 ；混合架構的 SSM 層 KV 頭數為 0 自然不計；滑動視窗層只計 min(上下文， 視窗) ，兩部分都乘並行數。

實例 SmolLM2-135M ： 30 層、每層 3 個 KV 頭、頭維度 64 ，算得每 token 23,040 位元組，與手算一致。

校正 模型載入成功後，代理自動量一次實際用量存起來；下次換回這顆，估算優先用實測值。

同樣 128K 上下文，線性注意力 0.13 GiB ，全注意力可能幾十 GiB ，不能套同一個數字。

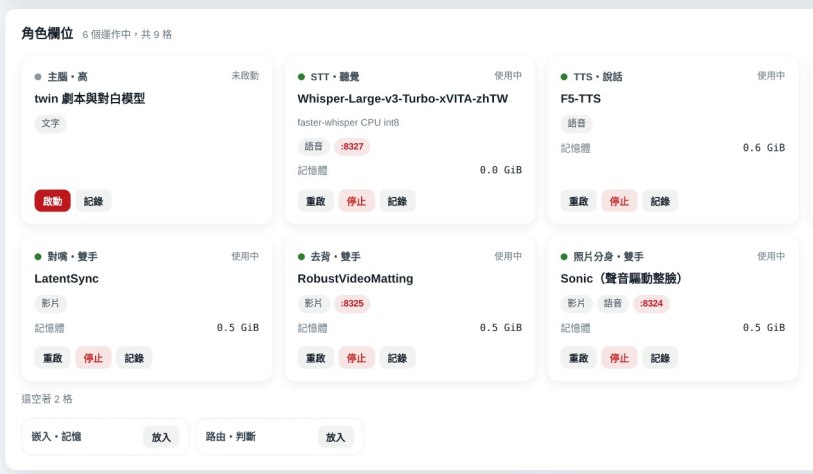
角色欄位與服務狀態：不把狀態壓成開或關

已實作且有測試 Jetson AGX Thor 與兩台 AI 395 的角色欄位在正式環境運作

欄位登錄 每台的 ~/.xconsole/registry.json ：角色、 systemd 服務名、引擎種類、預估需求、能力標籤、是否為關鍵服務。登錄檔只能經 SSH 寫入，沒有 API 可改。

狀態 直接取 systemd 的真實狀態： active 、 activating 、 deactivating 、 failed ；設了 Restart= 的服務退出時會停在 activating ，要再看 SubState 是否為 auto-restart 。

速度與用量 常駐記憶體取服務所有程序的 RSS ；速度取自引擎每次請求留下的紀錄，累積成區間，不是規格值。



正式環境： Jetson AGX Thor 的角色欄位

卡在啟動中的服務，壓成「關」或「開」都會誤導判斷。

換模型的流程：換一整份設定，打得通才算成功

已實作且有測試　失敗自動還原：整合測試與正式環境都驗證過

步	動作	細節
1	只接受掃描清單	Strata 要 pack（含 tokenizer）、權重與 strata-< 名稱 >.json 三樣齊； GGUF 要分片齊全、檔頭讀得出
2	備份與組設定	以該模型自己的設定為底，只沿用跟機器有關的項目，以及跟著欄位走的金鑰、輸出上限、自訂名稱
3	准入試算	用要換過去那顆的大小算，不沿用舊模型的估值
4	重啟	reset-failed 後 restart ；跨引擎時先停用舊服務並取消開機自啟，新服務設成開機自啟
5	驗證	等埠回得出 /v1/models 且模型名稱在清單內（設了金鑰時 401 也算服務活著）
6	失敗還原	寫回原設定、還原開機自啟狀態、重啟原服務並再驗一次；回報原因

判斷成功看的是模型真的回得出清單，不是指令有沒有回傳成功。

llama.cpp 引擎：代理替欄位產生設定與服務

已實作且有測試 test_llama_engine.py 36 項 + 真機冒煙測試 12 項（真的 llama-server ）

設定與服務 每個欄位一份 ~/.xconsole/engines/< 欄位 >-llama.json （ 0600 ）與 xc-< 欄位 >-llama.service ；服務的進入點是 xconsole_agent.py --launch-llama ，讀設定後 exec 成 llama-server ，不多一層程序。

選哪支 llama-server 登錄檔 machine.llama_server 優先；否則挑家目錄下版本號最大、有 GPU 後端（ HIP 、 CUDA 、 Vulkan ）的那支。

機器專屬參數 需要額外參數或環境變數時寫在 machine.llama_args 與 machine.llama_env ，不改程式。

固定帶上的參數

-c 上下文 × 並行、 -np 並行、 -ngl 999

--alias 欄位的模型名稱、 --jinja

--no-mmap ：統一記憶體機開 mmap 會多吃一份頁快取

--api-key-file ：金鑰用檔案交給引擎，不放在指令列

-n 單次輸出上限（有設才帶）

金鑰在檔案裡， ps 看不到。

跨引擎切換：同一個欄位、同一個埠， IDE 網址不用改

已實作且有測試 Strata→GGUF 、 GGUF→GGUF 、 GGUF→Strata 、壞模型還原都有測試

切過去 Strata→llama.cpp ：停掉 Strata 服務並取消開機自啟， llama.cpp 服務接手同一個埠並設成開機自啟； Strata 的服務與設定記在欄位的 engines 裡，之後切得回來。

跟著欄位走 API 金鑰、單次輸出上限、使用者自訂的模型名稱都隨欄位帶過去； llama.cpp 那邊清掉的輸出上限，切回 Strata 時也一併清掉。

能力標籤 「看圖」依新模型是否帶 mmproj 重算，不沿用上一顆。

失敗 任何一步失敗就整組還原：引擎、服務的開機自啟與開關狀態、設定檔、金鑰檔都回到原樣。

限制 96 GiB 的 AI 395 裝不下兩顆大模型，跨引擎時無法先在別的埠驗證再切，必須先停舊的釋放記憶體。

開機自啟一起換，不然重開機後兩個引擎會搶同一個埠。

模型庫：存放規則、下載與刪除

已實作且有測試 假 Hugging Face 整合測試 + 真 Hugging Face 冒煙測試（ 105 MB 約 6 秒下載完 ）

項目	做法
存放規則	< 存放區 >/< 資料夾 >/< 模型 >.gguf ；預設 ~/models/gguf ，可由 machine.gguf_root 改到別顆碟；分片齊才列入， mmproj 放同資料夾
多個量化	同一資料夾的多個量化各算一顆；分片不齊或檔頭讀不到的，清單會寫出原因
下載	列出 repo 的 GGUF 並依量化分組；推測解碼用的輔助檔標出並排到最後；一次只下載一個
續傳與核對	8 MB 分段寫入 .part ，中斷後以 Range 接續，大小核對才改名；資料夾名稱為「作者 -- 模型」
權杖與磁碟	Hugging Face 權杖只在下載期間放在記憶體，不寫任何檔案；下載前檢查剩餘空間並保留 10 GiB
刪除	只刪存放區第一層資料夾內的 GGUF ；有欄位正在使用的不准刪； Strata 家族不從這裡刪
中國來源	架構或名稱含 qwen 、 deepseek 、 glm 、 kimi 、 minimax 、 hunyuan 、 internlm 、 baichuan 、 doubao 、 ernie 等即標示

放錯位置的模型，清單會寫出缺什麼，不會默默消失。

API 串接：設定、驗證、範例與實測

已實作且有測試 test_api_integration.py 23 項；正式環境實測對話約 1.5 秒、工具呼叫約 2.3 秒

項目	做法
可改的設定	對外網址（不重啟）；模型名稱、上下文、單次輸出上限、金鑰（重啟後驗證，失敗換回）
輸入驗證	名稱限英數與 ._-:/ 且 1 ~ 80 字；網址限 http(s) ；上下文 2,048 ~ 1,048,576 ；輸出上限要小於上下文
金鑰	256 位元隨機值，只在產生的回應裡出現一次；套用後實際驗證「沒帶金鑰打對話端點回 401 」
範例	依目前設定產生 8 種： VS Code Copilot 、其他 IDE 、 OpenCode 、 Continue 、 Cline 、 Claude Code 、 curl 、 Python
本機實測	模型清單、沒帶金鑰被擋、一般對話、工具呼叫（給一個查時間的假工具，看模型有沒有呼叫）
經網路實測	從主控台打對外網址，不跟轉址；有金鑰時改打需要金鑰的對話端點驗證

llama.cpp 的模型清單本來就公開，驗金鑰一定要打對話端點。

情境切換與記錄檢視

已實作且有測試　寫程式後端、抓料清洗兩個情境在 AI 395 上使用

情境　同一顆模型用途不同時參數不同：寫程式的用戶端一輪送四萬 token 以上，要長上下文；抓料清洗每段只有幾千 token ，把並行讓出來換吞吐。情境寫在登錄檔 profiles ，切換時先備份設定、寫入參數、重啟，失敗還原上一版。情境只套用在 Strata 欄位，llama.cpp 欄位會略過並說明。

記錄　選一台機器、一個服務，直接看 systemd 記錄與引擎自己的記錄檔；區塊固定高度在區塊內捲動，不讓頁面無限延長。

情境	上下文	並行數	用途
寫程式後端	131,072	1	GitHub Copilot 等一次送大量程式碼脈絡
抓料清洗	32,768	2	網頁抓取後的資料清洗，每段提示較短

情境是一組參數，切換是一個按鈕，失敗是自動還原。

安全架構：逐層的做法

以資通系統防護基準中級、 OWASP ASVS L2 為基準

層面	做法	測試
傳輸	只提供 HTTPS （ TLS 1.2 以上 ）， http 打到同埠回 308 ；主控台連代理也走 HTTPS 並驗證對方憑證	安全測試第 8 節
認證	PBKDF2-SHA256 200,000 輪加鹽；同一來源 15 分鐘錯 5 次、全部來源錯 50 次即擋	安全測試第 9 節
會話	伺服器端票證、資料庫只存 SHA-256 ； HttpOnly 、 Secure 、 SameSite=Strict ； 12 小時	安全測試第 2 節
授權	所有 /api 預設要登入；機器與欄位 id 只能指到登錄檔內的項目	安全測試第 1 、 4 節
輸入	POST 只收 JSON （擋跨站表單）； 6 MB 上限，超過立即 413 ；路徑一律限定在目錄內	安全測試第 3 、 5 節
輸出	統一錯誤處理不回堆疊； CSP 無 unsafe-inline ，另送 COOP 、 COEP 、 CORP	安全測試第 7 節
機密	代理權杖 AES-256-GCM 加密存放；模型金鑰只顯示一次；稽核寫入前遮罩	安全測試第 10 節

每一層都有對應的測試， CI 每次修改都重跑。

認證與會話：被偷的票證、猜密碼、跨站請求

已實作且有測試　安全測試在修正前抓到 keep-alive 身分沿用問題

伺服器端票證　以前是無狀態簽章票證，登出只清瀏覽器 cookie ，被偷走的票證 12 小時內照樣能用；現在票證記在伺服器端，登出、換密碼即作廢，資料庫只存雜湊。

節流　同一來源 15 分鐘內錯 5 次就回 429 ，被擋期間連正確密碼也不放行，讓暴力猜測無從判斷是否猜中；不鎖帳號，避免內網任何人都能把管理者鎖在門外。

跨站請求　SameSite=Strict ，另外 POST 一律只收 application/json ；跨站的表單只能送 urlencoded 、 multipart 與 text/plain ，送不進來。

keep-alive　同一條連線上的多個請求共用同一個處理物件，身分快取曾被下一個請求沿用：登出後在同一條連線上仍是登入狀態。現在每個請求開始時清掉，並有測試守著。

這個漏洞是自己寫的攻擊式測試抓到的，修正前先確認測試會失敗。

主控台與節點代理之間的信任

已實作且有測試　沒有對方憑證就不連，不退回不驗證或明文

規劃中　節點端權杖改存雜湊（ G2 ）

連線　各機代理第一次啟動自產憑證；部署腳本經 SSH 取回當作信任依據存在主控台。代理送出的憑證跟當初信任的不同就拒絕連線；代理重裝後重跑部署腳本或在網頁匯入新憑證。

權杖　每台一把隨機 Bearer 權杖，只走 Authorization 標頭；網址帶權杖一律 401 。主控台端以 AES-256-GCM 加密存在 fleet.json ，啟動時自動把既有明文遷移成密文。

代理的入口　代理只接受登錄檔內的欄位與掃描清單內的模型；沒有改登錄檔的 API ，避免拿到權杖的人把記錄檔指到私鑰再讀出。

缺口　節點端的權杖檔目前以 0600 明文存放，能讀這個檔的人已擁有執行模型服務的帳號；計畫改為節點只存雜湊。

信任來自 SSH 取回的憑證，不是第一次連上就相信。

HTTPS 與憑證：熱換、檢查、還原

已實作且有測試 test_tls.py 36 項、 test_e2e_tls.py 21 項； HTTPS 交付檢查缺 0

熱換 不在監聽 socket 上先包 TLS ，改在處理執行緒裡才握手，每條連線當下讀目前的憑證物件；換憑證就是換掉那個物件，不重開服務。同時先看第一個位元組，不是 TLS 就 308 轉 https 。

套用與還原 套用後實際連自己一次核對送出的憑證指紋，不符就換回；保留最近 10 張可一鍵還原。

CSR 私鑰在伺服器上產生，以 AES-256-GCM 加密保存；廠商只寄回憑證時自動對上那把私鑰。

HSTS 只有正式憑證套用後才能開；剩 30 、 14 、 7 、 1 天各提醒一次。

接受的格式

PFX / P12 （含 Windows 匯出的 RC2-40 與 3DES 舊式加密）

PEM 、 CER （ DER 也行）、 CRT ，私鑰與中繼憑證順序不拘

P7B （搭配當初的私鑰或系統內產生的 CSR ）

匯入前檢查：成對、效期、主機名稱涵蓋、憑證鏈、金鑰強度、自簽

換憑證是管理者在網頁上做的事，不必叫工程師進主機。

操作紀錄、取樣與資料保存

已實作且有測試 稽核不含密碼、金鑰與錯誤的金鑰（安全測試與整合測試皆驗證）

資料	內容	保存	保護
稽核紀錄	時間、帳號、來源 IP 、動作、目標、結果，與遮罩後的細節	400 天	寫入前經 redact()
機器取樣	每 5 分鐘：各池用量、 GPU 功耗與使用率、欄位狀態、請求計數	400 天	不含個資
登入票證	票證的 SHA-256 、帳號、來源、到期時間	12 小時	只存雜湊
設定與機密	fleet.json 、 TLS 私鑰、 CSR 私鑰、主金鑰	持續	0700 / 0600 ，機密加密

遮罩規則 鍵名像機密的一律換成「已遮罩」（ password 、 token 、 key 、 pem 、 cookie 、 authorization… ），含 PRIVATE KEY 或 enc:v1: 的字串整段遮掉，長字串截斷；所有寫稽核的地方都經過同一支函式。

稽核最常見的洞是把整包請求寫進去，我們只寫遮罩後的必要欄位。

服務使用報告：數字由程式算，估算值帶著假設

已實作且有測試 test_usage_report.py 26 項； 2026 年 10 月 22 日產出第一份

區塊	回答的問題	怎麼算
誰在用	多少次登入、多少次操作	稽核紀錄
用了多少電	GPU 晶片本期耗電度數	取樣功耗 × 間隔；超過 15 分鐘的空窗不計
地端處理了多少	請求數、匯入與生成 token 、快取命中	代理累計計數器取差值；變小視為重啟
設備有沒有用到	在線比例、池平均與峰值、低於底線次數	每 5 分鐘取樣
連線與操作安不安全	憑證狀態、代理信任、准入擋下與強制略過	憑證管理與稽核
系統健不健康	待處理事項清單	以上全部

電價與雲端單價沒設，就只列度數與 token 量，不自己補數字換算金額。

品質與資安證據

已實作且有測試 GitHub Actions 七個作業全綠才能合併； CodeQL 高風險有規則擋合併

測試	通過	涵蓋
agent/test_admit.py	15	准入公式、 KV 、底線邊界、模型名稱
console/test_tls.py + test_e2e_tls.py	57	憑證匯入、熱套用、還原、轉址、經網頁走完整套
console/test_usage_report.py	26	報告六塊、計數器、空窗、遮罩
test/security/test_security.py	52	從攻擊者角度：票證、 IDOR 等價、 SSRF 、穿越、節流、標頭、 TLS
test/test_api_integration.py	23	API 設定、金鑰、自動還原、不跟轉址
test/test_llama_engine.py	36	模型庫、檔頭 KV 、跨引擎切換與還原、下載、刪除
console/test_ui_guard.py	5	彈窗不准點背景關閉、 Enter 擋輸入法

214 項自動化測試加 12 項真機冒煙測試，掃描與 SBOM 弱點皆為 0 。

技術規格總表

項目	規格
執行環境	Python 3.10 以上（正式環境 3.12 ）、 systemd 使用者服務；主控台以 venv 執行，依賴釘版本
依賴	cryptography 50.0.2（含 cffi 2.1.1 、 pycparser 3.11 ）；報表產生器另需 matplotlib 、 python-docx
埠	主控台 9400（ HTTPS ，同埠 http 轉址 ）；節點代理 9401（ HTTPS ）；模型依欄位
推論引擎	Strata 0.1.45（ Qwen3.8-Flash-Next 家族 ）； llama.cpp （自動挑本機最新、有 GPU 後端的建置 ）
已驗證的機器	AMD Ryzen AI 395（ HP Z2 Mini G1a ， ROCm 7.1.1 ）×2 、 Jetson AGX Thor（ JetPack 7.2 ）×1
瀏覽器	以 Chromium 系瀏覽器（ Chrome 、 Edge ）實測；版面在 1440 、 1280 與 375 寬都量過
原始碼	GitHub 公開倉庫 guessleej/xConsole ， main 分支受保護

版本與機型都寫實際用的，沒驗過的不列。

已知限制：照威脅模型缺口清單照實寫

項目	目前狀況	影響與因應
模型 API 走 http（ G1 ）	推論引擎本身不提供 TLS ，金鑰與提示在內網明文傳輸	畫面明示；計畫由代理加一層 HTTPS
節點端權杖明文（ G2 ）	~/.xconsole/token 以 0600 存放	計畫改為只存雜湊
管理者無兩步驟驗證（ G3 ）	單一管理者帳號，以節流與換密碼作廢補強	政府案可依要求補上
第三方滲透測試（ G4 ）	目前是開發團隊自測加自動化掃描	依合約安排
只有一種角色	沒有唯讀或分權帳號	主控台只給管理者用
單一 GPU 的顯存歸屬	無法逐程序區分，依各欄位預估比例分攤	多模型並存時為估計值
工具呼叫看模型	llama.cpp 的工具呼叫取決於模型的對話範本	用「實測一次」確認
DGX Spark	公式已自我檢查，未接實機	待實機驗證

技術長最需要的是誠實的限制清單。

路線圖：已排定的補強

以下皆為規劃中，尚未實作

2026 年 12 月 31 日	**模型 API 改走 HTTPS**	節點代理加 TLS 反向代理，沿用代理憑證與網頁換憑證流程， IDE 改打 https
2026 年 12 月 31 日	**節點端權杖只存雜湊**	部署腳本產生權杖後只交給主控台，節點只留雜湊比對
2026 年 12 月 31 日	**設定與資料加密備份**	~/.xconsole 以公鑰加密納入機群備份
2027 年 3 月 31 日	**管理者兩步驟驗證**	TOTP ；交付政府案前完成或以客戶網段限制替代
待排	**DGX Spark 實機驗證**	接上實機確認單一池計算與 nvidia-smi 的回報

有日期的才叫路線圖。

導入需求：貴單位要準備什麼

項目	需求
機器	要納管的 AI 機（ AMD AI 395 、 Jetson / MIC-743 、 DGX Spark 或多卡伺服器），Ubuntu（已驗證 24.04 ）
帳號	每台一個一般使用者帳號，可用 systemd 使用者服務；部署時需要 SSH 連線
網路	瀏覽器能連主控台 9400 ；主控台能連各機 9401 ；同仁的電腦能連模型的埠
憑證	正式憑證或內部 CA ；沒有時先用暫用自簽，之後在網頁上換
模型	要提供給同仁的模型清單；政府、學校、醫院案只安裝非中國來源的模型
人	主控台管理者一位；資安聯絡窗口一位

不需要另外買管理機，主控台可以裝在其中一台 AI 機上。

導入步驟：四個指令，每一步都有驗證

步	做什麼	怎麼做	驗證
1	裝節點代理	每台執行 deploy_agent.sh（第一次加 --seed 寫入欄位登錄）	代理的健康檢查以 HTTPS 回應
2	裝主控台	deploy_console.sh ：建 venv 、組納管清單、取回各機代理憑證	http 打主控台回 308
3	設帳密	xconsole_server.py --set-credentials ，帳密由標準輸入讀，不進指令列與環境變數	網頁登入
4	換憑證與交付	網頁上傳正式憑證；跑安全、 HTTPS 、機密與使用報告四項檢查	四項檢查皆通過

單機模式用 deploy_standalone.sh 一行完成。



Thank You

xConsole 模型主控台　技術簡報

云碩科技股份有限公司　xCloudinfo Corp. Limited

資安聯絡窗口　security@cloudinfo.com.tw